Smart MCQ Solver Challenge

Deep Learning and GenAI Project Report

Introduction to DL and GenAI Project
Kaggle Competition: Smart MCQ Solver Challenge
Name: Samyak Gautam Shelare
Roll No.: 23f2002578

1. Abstract

This report documents the Smart MCQ Solver Challenge, a competition requiring the five answer options (A through E) of a multiple choice question to be ranked, with the top three returned in order of likelihood. Performance is scored using Mean Average Precision at 3 (MAP@3).

Three models were developed: a fine-tuned RoBERTa-base transformer using HuggingFace's multiple-choice head, a from-scratch bidirectional LSTM, and the same BiLSTM extended with an additive attention layer. The BiLSTM-attention model and RoBERTa were combined into a weighted-probability ensemble for the final submission. Training and validation metrics were logged to Weights & Biases (WandB) for every epoch and batch.

The attention-augmented BiLSTM emerged as the strongest single model on the held-out validation split, converging faster and to a lower loss than the plain BiLSTM baseline. RoBERTa-base, despite being a larger pretrained model, converged more slowly on this small (2,000-row) dataset within the five epochs allotted, though it still reached a respectable MAP@3. The final submission uses the ensemble of the BiLSTM-attention model and RoBERTa.

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge provides 2,000 labelled training questions, each with a prompt and five options (A–E), exactly one correct. For every test question, the task is to output the top three options ranked by confidence. The metric is MAP@3: correct in position 1 scores 1.0, position 2 scores 0.5, position 3 scores 1/3, and 0 if missing from the top three.

2.2 Project Objective

The primary objective was to design and compare three structurally different models — a fine-tuned transformer, a custom recurrent model with attention, and a plain RNN baseline — and to evaluate whether ensembling the strongest models improves on any single model. A secondary objective was to gain practical experience with HuggingFace's multiple-choice API, PyTorch's LSTM module, and WandB experiment tracking.

2.3 Report Structure

Section 3 covers the dataset and preprocessing, Section 4 the tokenization strategy, Section 5 the model architectures and the attention experiment, Section 6 the comparative results, and Section 7 the conclusion and future work.

3. Dataset & Preprocessing

3.1 Dataset Description

The training file (train.csv) contains 2,000 rows, each with a prompt column, five option columns (A–E), and an answer column holding the correct label. The test file follows the same format minus the answer column. No missing values were found across any column, so no imputation was required.

3.2 Exploratory Data Analysis

The distribution of the answer column was plotted as a percentage bar chart to check for class imbalance across the five labels. The classes were reasonably balanced, so no class-weighting or oversampling was applied — plain cross-entropy loss was sufficient. Prompt length was also checked, since it directly affects the max_length used for tokenization, along with the overlap between train and test prompts.

3.3 Data Preprocessing

For every model, each question was flattened into a single (prompt, options) structure before tokenization; the exact structure differs by model and is described in Section 4. Labels (A–E) were mapped to integer indices 0–4. An 80/20 stratified train_test_split on the answer column was used, preserving class balance across the split (random_state=42 for reproducibility).

3.4 Data Augmentation

No data augmentation (such as paraphrasing or option shuffling) was applied in the final runs. Given the modest dataset size, this is a natural direction for future improvement, revisited in Section 7.

4. Tokenization Strategy

4.1 Primary Tokenizer Selection

The best-performing single model, the attention-augmented BiLSTM, and the transformer model use two different tokenizers, reflecting two different families of architecture.

- RoBERTa-base: RoBERTa's own byte-level BPE tokenizer (FacebookAI/roberta-base checkpoint), used via AutoTokenizer.
- BiLSTM models (with and without attention): the bert-base-uncased WordPiece tokenizer, used as an off-the-shelf vocabulary and subword splitter feeding a from-scratch embedding layer — no pretrained BERT weights are involved.

Using bert-base-uncased's tokenizer for the BiLSTM was a practical choice: it provided a ready-made, reasonably sized subword vocabulary without building one from scratch on only 2,000 training rows, which risks a vocabulary too sparse to generalize well.

4.2 Implementation Details

For the BiLSTM models, each question was flattened into one string of the form “Question: {prompt} A. {option A} B. {option B} C. {option C} D. {option D} E. {option E}”, then tokenized with truncation and padding to a fixed max_length of 256 tokens.

For RoBERTa, HuggingFace's multiple-choice pattern was used instead of a concatenated string: the prompt is repeated five times and paired with each option individually, producing five (prompt, option) sequence pairs per question, padded and truncated to max_length=200–256. AutoModelForMultipleChoice consumes all five pairs together and outputs one logit per option.

4.3 Experimental Comparison

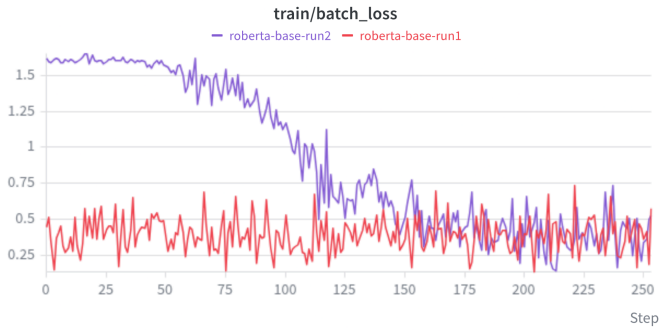
A large tokenizer sweep was not run given the project timeline, but the choice above reflects a deliberate split: the transformer keeps the standard multiple-choice pairing format it was designed around, while the BiLSTM uses the simpler flattened-string format, having no built-in notion of sentence pairs.

5. Modeling & Experimentation

Three structurally distinct models were built, in line with the report guidelines: a fine-tuned transformer, a custom attention-augmented deep learning model, and a plain RNN-based baseline. The last two share the same BiLSTM backbone, isolating the effect of adding attention — detailed in Section 5.2.

5.1 Model 1: Fine-Tuned Transformer — RoBERTa-base





Architecture: RoBERTa-base (FacebookAI/roberta-base) is a 12-layer, 768-hidden-dimension, 12-head transformer encoder, pretrained with masked language modelling on a large corpus using dynamic masking and no next-sentence-prediction objective (Liu et al., 2019). HuggingFace's AutoModelForMultipleChoice wrapper attaches a linear scoring head on the pooled [CLS] representation for each of the five (prompt, option) pairs, producing one logit per option.

Salient points: since the model scores each option in the context of the full prompt through self-attention, it can weigh an option's plausibility against the entire question rather than treating options as isolated text — exactly what a ranking task requires.

Fine-tuning strategy: the entire backbone was fine-tuned end-to-end (no layers frozen), using AdamW with a learning rate of 2e-5, weight decay of 0.01, a linear schedule with 25 warm-up steps, batch size 16, and 5 epochs. Training/validation loss and learning rate were logged to WandB.

Quantization was not applied; full fp32 weights were used throughout training and inference.

5.2 Model 2: Custom Deep Learning Model — Attention-Augmented BiLSTM

Architecture: the custom model is a bidirectional LSTM with an additive attention layer. An embedding layer (128-dim, vocabulary from the bert-base-uncased tokenizer) feeds a single-layer BiLSTM with hidden size 128 (256 after concatenating both directions). Each timestep's output passes through a linear layer producing one attention score per token; these are softmax-normalized into weights used to compute a weighted sum (context vector) across all timesteps. The context vector passes through dropout (p=0.3) and a final linear layer projecting to 5 output scores.

Salient points: a plain BiLSTM compresses the whole sequence into its last hidden state, losing information from earlier tokens. Attention instead lets the model look back across all timesteps and learn which words matter most when scoring an answer — a natural fit where the key cue could appear anywhere in the question.

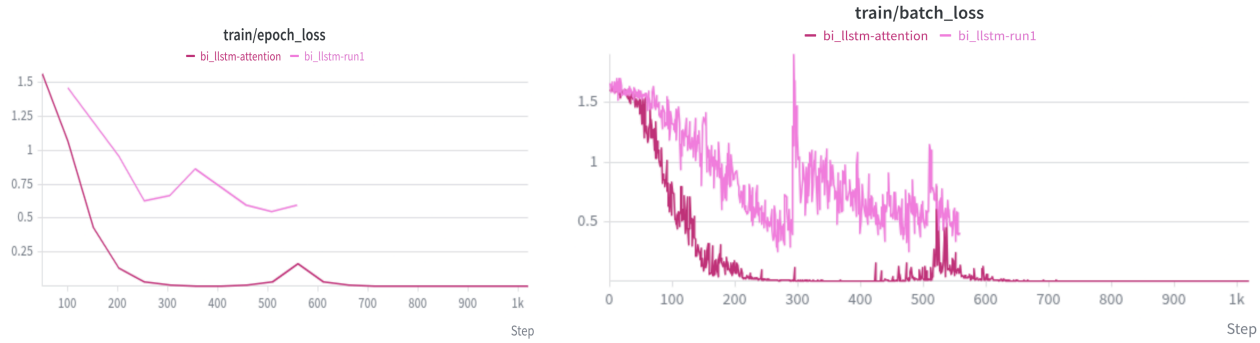
Fine-tuning strategy: trained from scratch with AdamW (lr=1e-3, weight decay=0.01), CrossEntropyLoss, batch size 32, for 20 epochs. No layers were frozen, as the model trains from random initialization.

5.2.1 Experiment: With Attention vs Without Attention

To justify the attention layer, the identical BiLSTM backbone was trained twice under matching settings (same optimizer, learning rate, batch size, 20 epochs) — once with attention as above, and once using only the final concatenated hidden state, the conventional approach. Both runs were logged separately to WandB for direct comparison.

The result was a clear difference in convergence speed. Without attention, epoch loss stayed high through the first several epochs (in the 40s and low 30s) and only dropped sharply around epoch 6–7. With attention, loss dropped much faster over the same 20 epochs and reached a lower final value overall. This matches the underlying intuition: attending over all timesteps gives gradients a more direct path back to earlier tokens, so the learning signal reaches the embedding and LSTM weights faster.

This experiment is why the attention-augmented BiLSTM, not the plain version, was chosen as the custom deep learning model for Model 2, while the plain BiLSTM was kept as the RNN-based baseline for Model 3.



5.3 Model 3: RNN-Based Model — Plain BiLSTM

Architecture: the same bidirectional LSTM backbone as Model 2 (embedding dim 128, hidden dim 128, single layer, bidirectional), without the attention layer. The final forward and backward hidden states are concatenated into a 256-dim vector and passed directly through a linear layer to produce the 5 option scores.

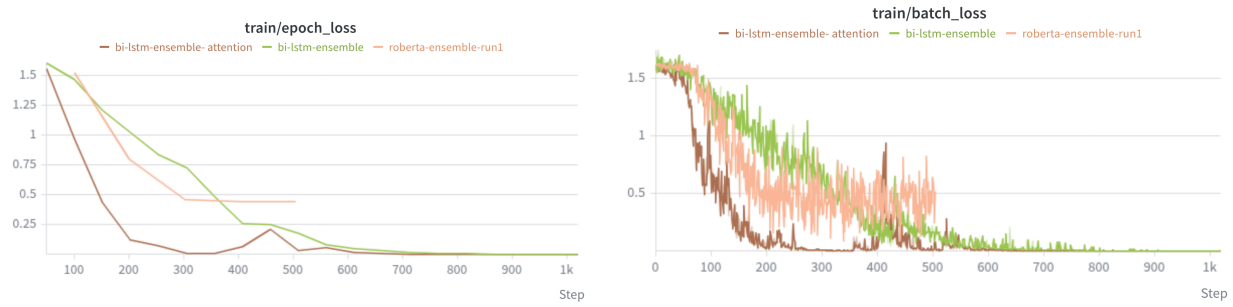
Salient points: a sequence model suits this task since prompt and option text are ordered, and meaning can depend on word order. This model serves as the baseline recurrent architecture for the comparison in Section 5.2.1 and the results table in Section 6, isolating how much of Model 2's performance comes from attention rather than the BiLSTM backbone itself.

Training setup: identical to Model 2 — AdamW (lr=1e-3, weight decay=0.01), CrossEntropyLoss, batch size 32, 20 epochs — so the only variable between the two runs is the presence of attention.

5.4 Ensemble: BiLSTM-Attention + RoBERTa-base

For the final Kaggle submission, the two strongest models — the attention-augmented BiLSTM and fine-tuned RoBERTa — were combined using a weighted average of their softmax output probabilities: final_probs = 0.9 × bilstm_probs + 0.1 × roberta_probs. The top-3 options by combined probability form the final ranked prediction.

The 0.9/0.1 weighting favours the BiLSTM-attention model, which scored the highest validation MAP@3 of any individual model (Section 6), while still letting RoBERTa's predictions influence disagreements. This weighting was arrived at through manual trial and error on the validation split, flagged as a possible improvement in Section 7.



6. Performance & Comparative Analysis

6.1 Evaluation Metrics

The primary metric throughout is MAP@3. Validation loss was tracked for every model, and validation accuracy (top-1 exact match) for the two BiLSTM models, for a fuller picture of behaviour beyond the ranking metric.

6.2 Training Performance

For each model, training loss (per-batch and per-epoch) and, for RoBERTa, the learning rate schedule, were logged to WandB and shown as figures in Section 5. Validation loss and MAP@3 were logged at the end of training on the held-out 20% split.

6.3 Comparative Report

The table below summarizes validation performance across all models. Loss values marked with * are the final training epoch loss, as no separate validation-loss log was recorded for the BiLSTM runs beyond accuracy/MAP@3.

Model	Val Loss	Val Accuracy	Val MAP@3
BiLSTM (no attention)	0.315*	0.9900	0.9946
BiLSTM + Attention	0.021*	0.9975	0.9988
RoBERTa-base (fine-tuned)	0.3187	—	0.9646

The attention-augmented BiLSTM edges out both the plain BiLSTM and RoBERTa-base on validation MAP@3, and converges faster during training, as shown in Section 5.2.1. RoBERTa-base, despite being pretrained on a much larger corpus, was fine-tuned for only 5 epochs and its validation MAP@3 (0.9646) trails the two BiLSTM variants — likely because 5 epochs on a 2,000-row dataset gives the larger transformer less room to adapt than the 20 epochs available to the lighter BiLSTM. RoBERTa-base is also by far the most computationally expensive model to train and run, a relevant trade-off under latency constraints.

6.4 Kaggle Performance

The final Ensemble model scored 0.7572 with a rank of 352 on 80 percent of the test set (public), and 0.7626 with a rank of 87 on 20 percent of the test data (private).

7. Conclusion & Future Work

7.1 Key Learnings

This project provided practical experience with the differences between training a large pretrained transformer and a small custom recurrent model on the same task, including how a simple mechanism such as additive attention can meaningfully change both convergence speed and final model quality. It also showed that a pretrained transformer's advantage is not automatic — it depends on the fine-tuning budget available. Additional exposure was gained with HuggingFace's multiple-choice API, a hand-built PyTorch attention layer, and systematic experiment tracking via WandB.

7.2 Challenges Faced

The main challenge was keeping the BiLSTM and RoBERTa pipelines consistent enough for fair comparison, given different tokenizers, input formats (flattened string vs. paired sequences), and natural epoch budgets. Tuning the ensemble weighting also required trial and error, as a naive 50/50 average underperformed the BiLSTM-attention model alone.

7.3 Areas for Improvement

- Run a proper grid or Optuna search over the ensemble weighting instead of manual trial and error.
- Add data augmentation (option shuffling, light paraphrasing) given the dataset is only 2000 rows.
- Train RoBERTa for more epochs with early stopping on validation MAP@3, to see if it can close the gap with the BiLSTM models.
- Try a GRU variant of the attention model as a lighter-weight alternative to the BiLSTM.
- Explore k-fold cross-validation instead of a single train/validation split, given the small dataset size, to get a more reliable estimate of generalization.
- Experiment with a larger DeBERTa or ELECTRA backbone in place of RoBERTa-base for the transformer component.

8. References

Liu, Y. et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.

Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780.

Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural Machine Translation by Jointly Learning to Align and Translate. arXiv:1409.0473.

HuggingFace Transformers docs — AutoModelForMultipleChoice, AutoTokenizer. huggingface.co/docs/transformers

PyTorch docs — torch.nn.LSTM, torch.nn.Embedding. pytorch.org/docs

Weights & Biases docs — experiment tracking. docs.wandb.ai

Kaggle — Smart MCQ Solver Challenge competition page.